



AWS Academy Cloud Developing
Module 13 Student Guide
Version 2.0.6

200-ACCDEV-20-EN-SG

© 2024, Amazon Web Services, Inc. or its affiliates. All rights reserved.

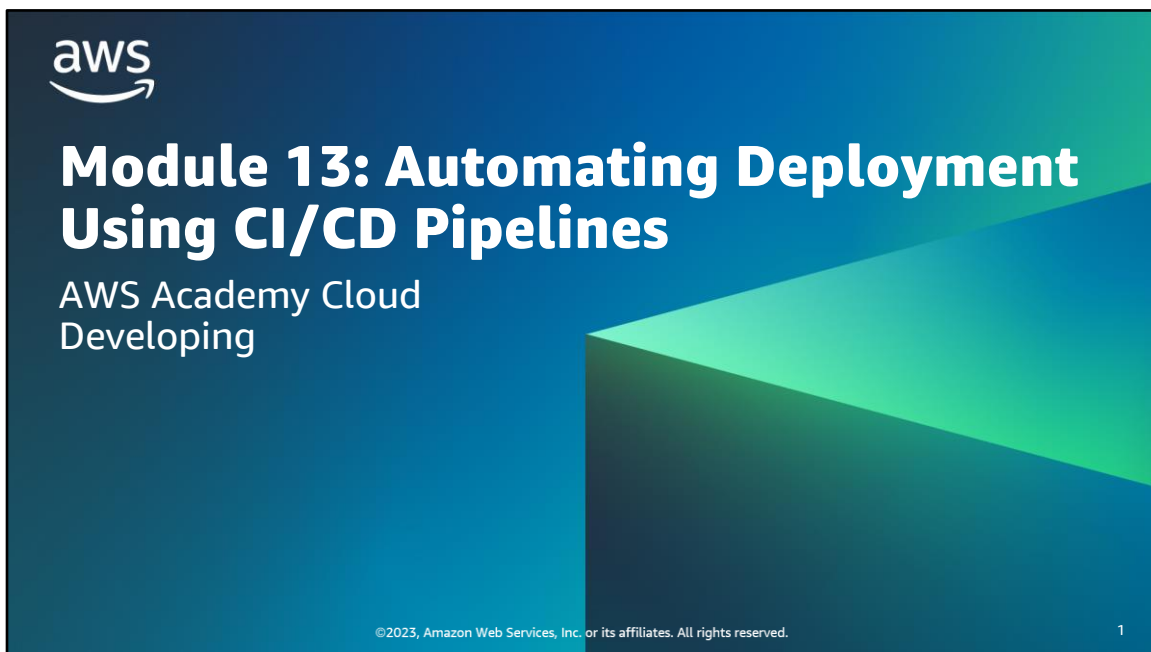
This work may not be reproduced or redistributed, in whole or in part,
without prior written permission from Amazon Web Services, Inc.
Commercial copying, lending, or selling is prohibited.

All trademarks are the property of their owners.

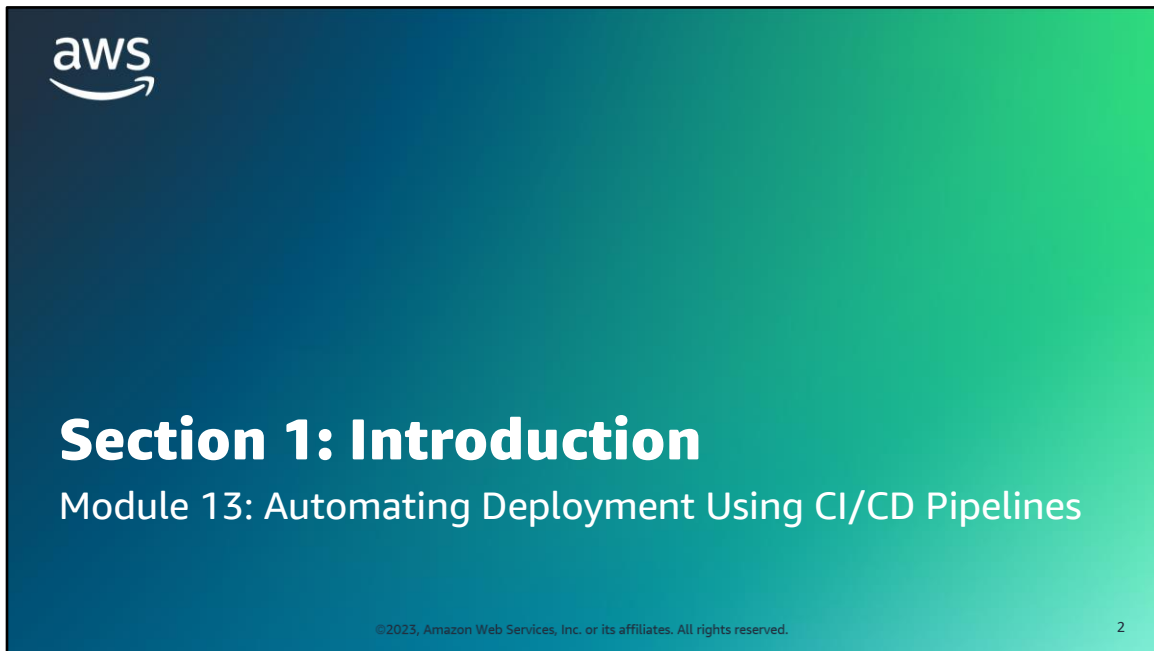
Contents

[Module 13: Automating Deployment Using CI/CD Pipelines](#)

4



Welcome to Module 13: Automating Deployment Using CI/CD Pipelines.



Section 1: Introduction

Module objectives

At the end of this module, you should be able to do the following:

- Describe DevOps
- Recognize AWS code services for continuous integration and continuous delivery (CI/CD)
- Describe how AWS CloudFormation is used to deploy applications
- Describe how the AWS Serverless Application Model (AWS SAM) is used to deploy serverless applications



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

3

At the end of this module, you should be able to do the following:

- Describe DevOps
- Recognize AWS code services for continuous integration and continuous delivery (CI/CD)
- Describe how AWS CloudFormation is used to deploy applications
- Describe how the AWS Serverless Application Model (AWS SAM) is used to deploy serverless applications

Module overview

Sections

1. Introduction
2. Introducing DevOps
3. Using AWS code services for CI/CD
4. Deploying applications with CloudFormation
5. Deploying serverless applications with AWS SAM

Lab

- Automating Application Deployment Using a CI/CD Pipeline



Knowledge check



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

4

This module includes the following sections:

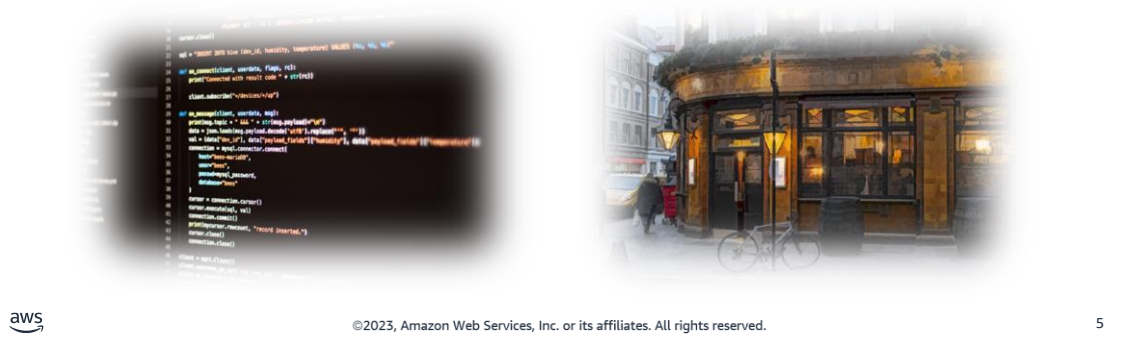
1. Introduction
2. Introducing DevOps
3. Using AWS code services for CI/CD
4. Deploying applications with CloudFormation
5. Deploying serverless applications with AWS SAM

This module also includes a lab about automating application deployments using a CI/CD pipeline.

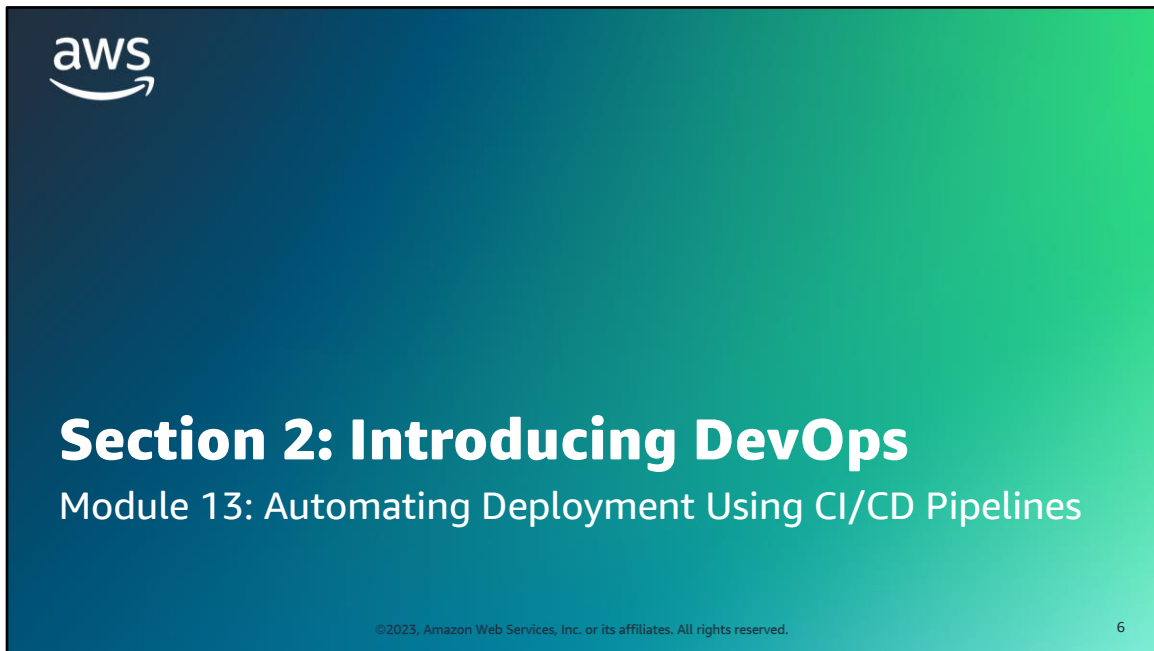
Finally, you will complete a knowledge check to test your understanding of key concepts covered in this module.

Café business requirement

Sofía wants to make the release process less prone to human error. She was chatting with Mateo, an AWS SysOps engineer, and he explained that DevOps is a philosophy and a set of practices and tools that support cloud application development, including automation of the development and release process.



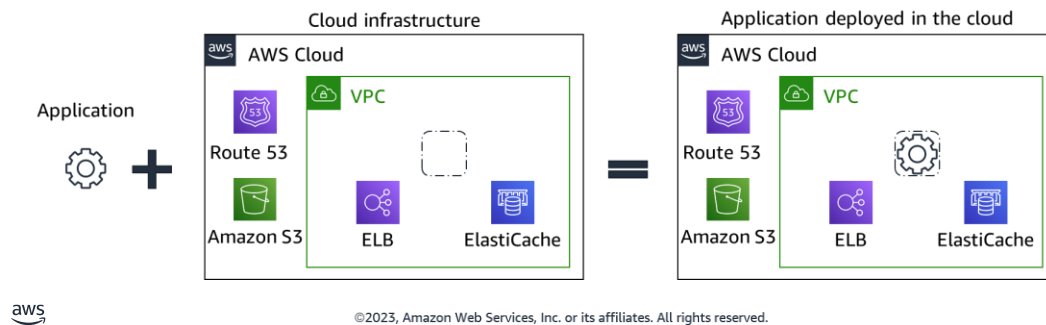
Sofía wants to make the release process less prone to human error. She was chatting with Mateo, an AWS SysOps engineer, and he explained that DevOps is a philosophy and a set of practices and tools that support cloud application development, including automation of the development and release process.



Section 2: Introducing DevOps

Deploying applications and infrastructure

- In the cloud, your application is not just your application. Your application is the application plus all of the associated infrastructure.
- DevOps is a combination of cultural philosophies, practices, and tools that support this method of application development.



In the cloud, *your application is not just your application*. Your application is your application *plus* the infrastructure required to run it. This can include virtual private clouds (VPCs), Elastic Load Balancing, Auto Scaling groups, Amazon Relational Database Service (Amazon RDS) databases, Amazon Simple Storage Service (Amazon S3) buckets and their associated data, Amazon ElastiCache servers, and any other AWS and non-AWS resources that your application requires to run.

This is a different way of thinking than some developers might be used to. Until the past 20 years or so, software development (Dev) and system operations (SysOps) teams often worked independently. Developers ensured that applications worked in their dev environments but left it to system administrators to set up the corresponding production environment. This disconnected approach resulted in expensive, time-consuming conversations between the two teams as SysOps teams struggled to host whatever the Dev teams created and do it in a scalable, secure manner.

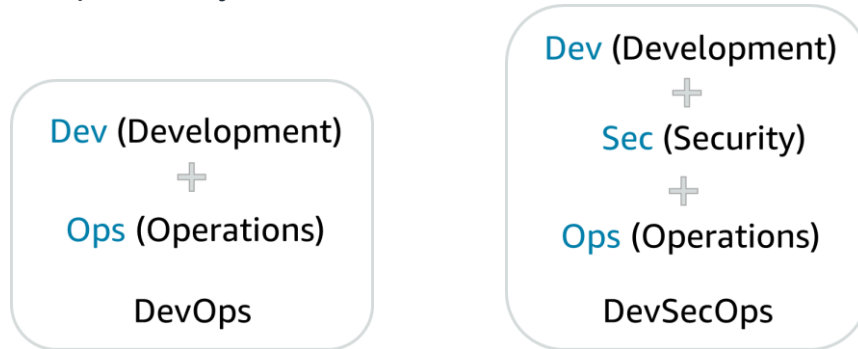
However, the advent of cloud technology changed all of that. Cloud technology gives developers the ability to provision their own infrastructure, test their code on that infrastructure at scale, and turn that infrastructure off when it's not in use. Developers can create environments that are similar or identical to the environments that their applications run in when deployed to production.

This technological advance has led to new ways of thinking about software development (for example, "In the cloud, your application is your application plus all of the associated infrastructure"), which in turn, has fostered new software development methodologies such as Agile, DevOps, and DevSecOps.

DevOps (that is, the combination of Development and Operations) is a combination of cultural philosophies, practices, and tools. DevSecOps takes this combination one step further by integrating security into DevOps.

Cultural philosophy of DevOps

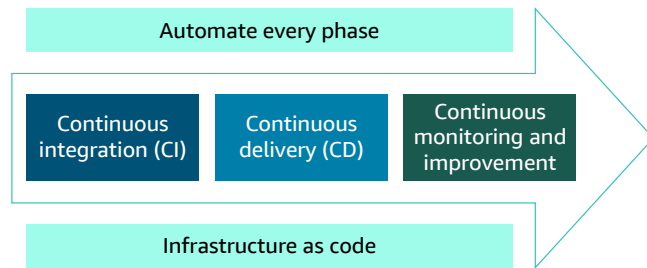
- Motto: People over process over tools
- Remove barriers between development and operations
- Shared responsibility



One motto in DevOps culture is "People over process over tools." DevOps is more of a *mentality* than a specific set of tools that you create. DevOps is about removing the barrier between development and operations teams, and getting them to communicate with each other. With DevOps, teams work together to optimize both the productivity of developers and the reliability of operations.

DevOps practices

- Microservice architecture
- Continuous integration and continuous delivery (CI/CD)
- Continuous monitoring and improvement
- Automation focused
- Infrastructure as code



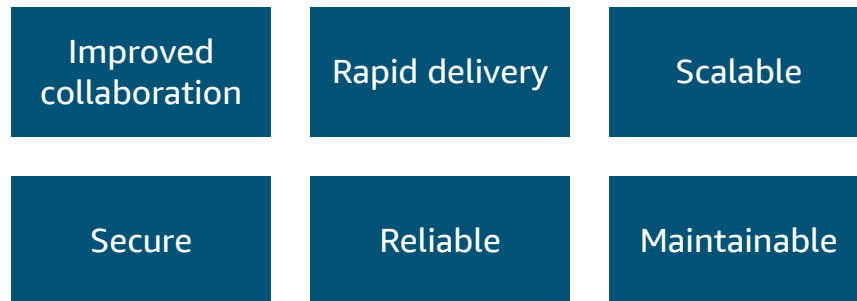
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

9

DevOps supports a set of practices that leads to rapid delivery of secure, reliable, and maintainable software. Those practices include the following:

- *Microservice architecture*: Instead of a monolithic application in which all processes are tightly coupled and run as a single service, you can write your applications as discrete microservices that run independently inside containers or as AWS Lambda functions. This leads to code that is more maintainable and scalable, because you can add, remove, or update individual application components as new features and updates are needed, rather than rewrite an entire code base.
- *Continuous integration and continuous delivery (CI/CD)*: Continuous integration (CI) is a software development practice in which developers regularly merge their code changes into a central repository. Continuous deployment (CD) is a software development practice where code changes are automatically built, tested, and prepared for production release. CD expands on CI by deploying all code changes to a testing environment, production environment, or both after the build stage is complete. When CD is properly implemented, developers always have a deployment-ready build artifact that has passed through a standardized test process. Code that has been deployed requires continuous monitoring and improvement. You will learn more about CI/CD in another section.
- *Automate everything*: DevOps supports the practice of automating every phase of the software development lifecycle. Automation reduces the level of human effort required in each phase and reduces errors caused by manual processes. In turn, business value can be delivered more quickly from the technical side to the company. This automation from beginning to end is an element of continuous deployment.
- *Infrastructure as code*: You can use CloudFormation to create a repeatable method to deploy your cloud architecture.

Benefits of DevOps



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

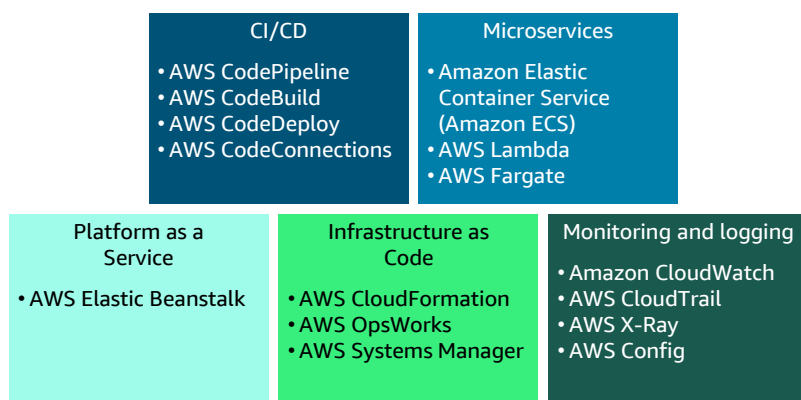
10

DevOps offers many benefits. Under DevOps, development and operations are no longer separate functions. These functions are often merged under a single team of engineers who work together across the entire software development lifecycle. These engineers develop a range of cross-functional skills. Quality assurance and security teams may also become more tightly integrated with development and operations throughout the software development lifecycle.

DevOps practices result in code that is secure, reliable, and maintainable. You can build in automation to test for bugs, and monitor and audit changes to your environment. Releases are more reliable because you can automate the creation of development environments that mirror production environments, which means you don't run into the problem of "It worked on my machine."

Using a microservice architecture for your applications makes code more maintainable and scalable, because you can add, remove, and exchange individual application components as new features. This means only updates are needed, instead of rewriting an entire code base. In turn, organizations that use DevOps can deliver value to their customers more quickly than organizations that use separate software development and infrastructure management processes. This speed means that businesses can scale to keep up with the pace of change and compete more effectively in the marketplace.

DevOps tools



In this module, you will learn about the AWS code services that support CI/CD, as well as the **AWS CloudFormation** service.



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

11

AWS offers several services to help you to provision and manage infrastructure, deploy application code, automate software release processes, and monitor application and infrastructure performance.

In this module, you will learn about the AWS code services that support CI/CD, as well as the AWS CloudFormation service.

Section 2 key takeaways



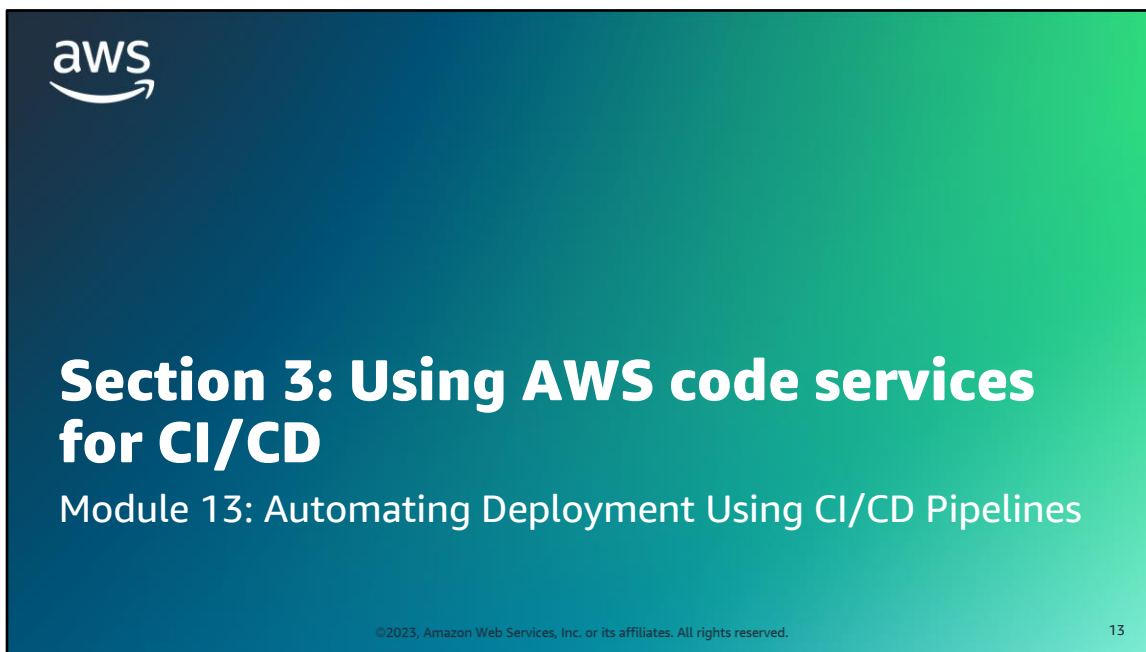
- In the cloud, your application consists of the application **plus** all of the associated infrastructure.
- DevOps is about removing the barrier between **development** and **operations** teams, and getting them to communicate with each other.
- DevOps practices result in code that is **secure**, **reliable**, and **maintainable**.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

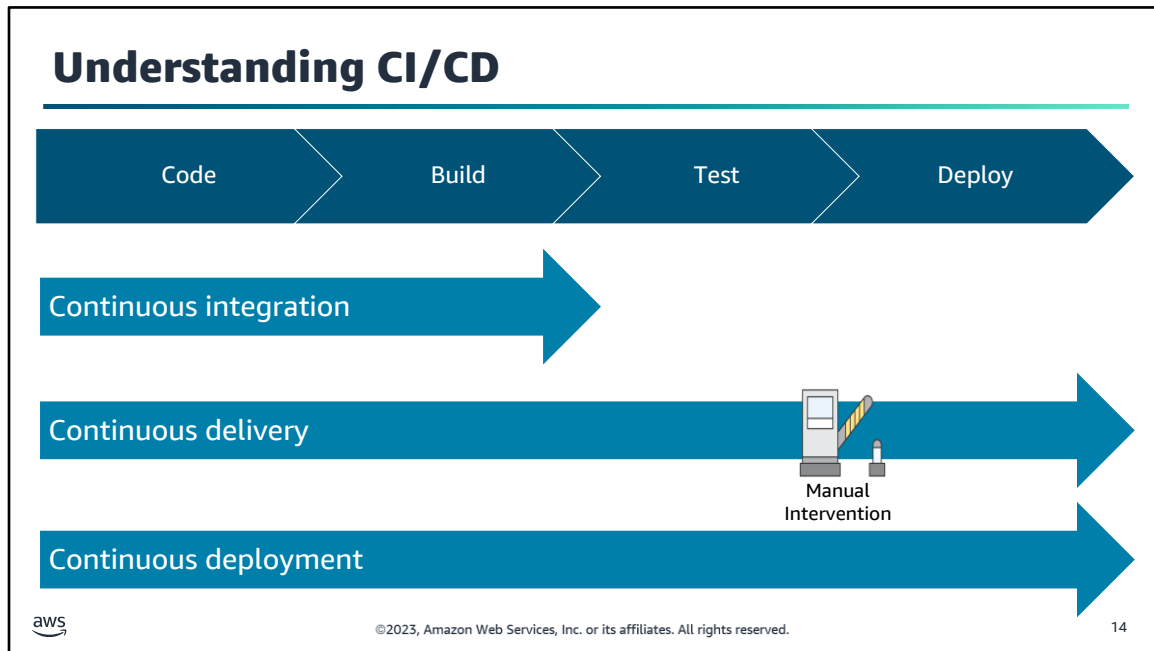
12

The following are the key takeaways from this section of the module:

- In the cloud, your application consists of the application *plus* all of the associated infrastructure.
- DevOps is about removing the barrier between development and operations teams, and getting them to communicate with each other.
- DevOps practices result in code that is secure, reliable, and maintainable.



Section 3: Using AWS code services for CI/CD



Continuous integration and continuous delivery (CI/CD) is a DevOps practice that spans the develop (that is, code, build, and test) and deploy stages of the software development lifecycle (SDLC).

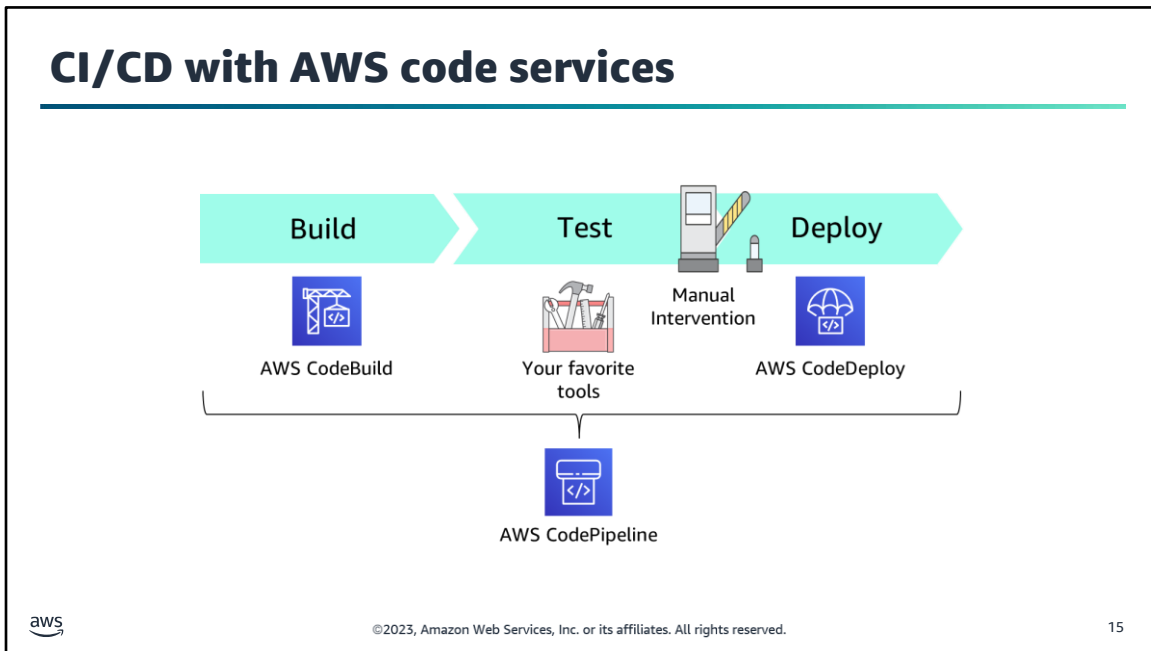
Continuous integration is the practice of continuously checking your code into a central code repository (for example, a Git repository) and verifying each change with an automated build and test process. Developers previously worked in isolation for an extended period and only merged their code changes into the mainline (or parent) branch after their feature was completed. Batching changes made it difficult to merge the business logic and also the test logic; however, continuous integration practices have made teams more productive and allowed them to develop new features faster. Continuous integration requires teams to write automated tests that improve the quality of the software being released and reduce the time that it takes to validate that the new version of the software is good.

Although there are different definitions of continuous integration, it is commonly thought that CI stops at the build stage.

Continuous delivery extends continuous integration to include testing out to production-like stages and running verification testing against those deployments. Continuous delivery might extend all the way to production deployment, but some form of manual intervention occurs between when code is checked in and when that code is released to customers.

Continuous delivery is a big step forward over continuous integration because it allows teams to gain a greater level of certainty that their software will work in production.

Continuous deployment is the automated release of software to customers. This automation extends from check-in through production without human intervention. Continuous deployment reduces the time that it takes to deliver business value (for example, new features or updates) to customers and, in turn, the time that it takes to get customer feedback on those new releases. This fast customer feedback loop helps you to iterate and release valuable software updates quickly to customers.



AWS code services can help you manage and automate your CI/CD pipeline. Those code services include the following:

- *AWS CodeBuild* is a fully managed, continuous integration service that compiles source code, runs tests, and produces software packages that are ready to deploy. With CodeBuild, you don't need to provision, manage, and scale your own build servers. CodeBuild scales continuously and processes multiple builds concurrently, so that your builds are not left waiting in a queue.
- *AWS CodeDeploy* is a fully managed deployment service that automates software deployments to a variety of compute services such as Amazon Elastic Compute Cloud (Amazon EC2), AWS Fargate, AWS Lambda, and your on-premises servers. CodeDeploy makes it easier for you to rapidly release new features, helps you avoid downtime during application deployment, and handles the complexity of updating your applications. You can use CodeDeploy to automate software deployments, which eliminates the need for error-prone manual operations. The service scales to match your deployment needs.
- *AWS CodePipeline* is a fully managed, continuous delivery service that helps you to automate your release pipelines for fast and reliable application and infrastructure updates. CodePipeline automates the build, test, and deploy phases of your release process every time there is a code change, based on the release model that you define. This helps you to rapidly and reliably deliver features and updates. You can easily integrate CodePipeline with third-party services such as GitHub or with your own custom plugin. With CodePipeline, you only pay for what you use. The service does not have any upfront fees or long-term commitments.

For more information about these AWS code services, see the product pages:

- AWS CodeBuild: <https://aws.amazon.com/codebuild/>.
- AWS CodeDeploy: <https://aws.amazon.com/codedeploy/>.
- AWS CodePipeline: <https://aws.amazon.com/codepipeline/>.

Section 3 key takeaways



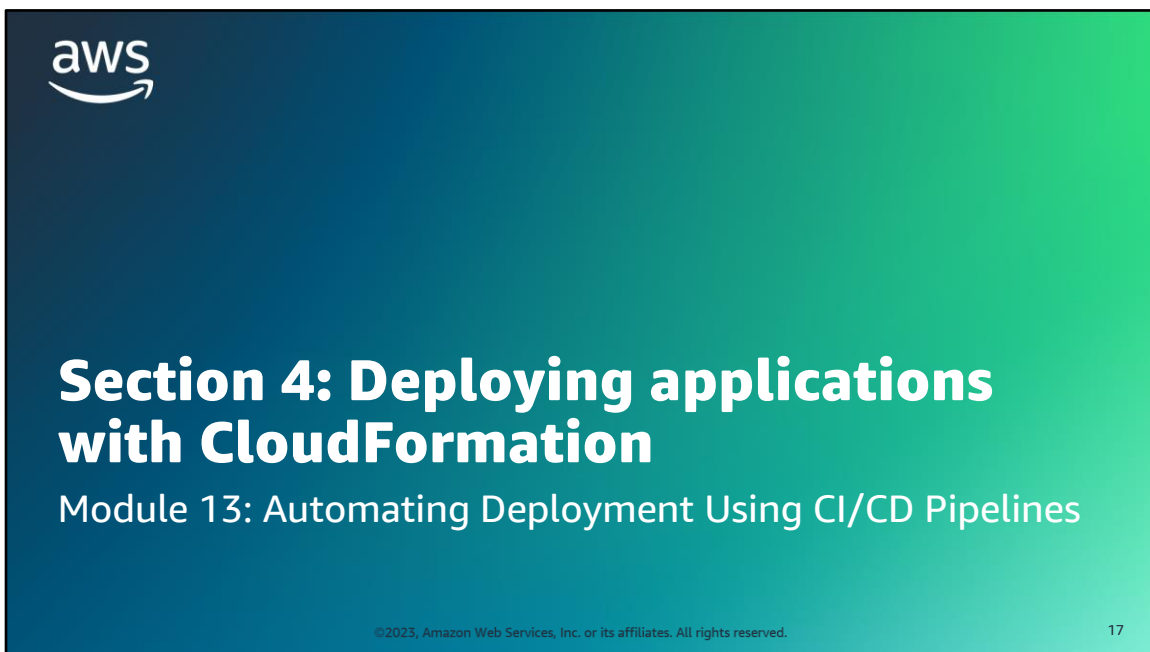
- CI/CD spans the **develop** and **deploy** stages of the software development lifecycle.
- It is commonly thought that continuous integration stops at the **build stage**.
- Continuous delivery **improves** on continuous integration by helping teams to gain a **greater level of certainty** that their software will work in production.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

16

The following are the key takeaways from this section of the module:

- CI/CD spans the develop and deploy stages of the software deployment lifecycle.
- It is commonly thought that continuous integration stops at the build stage.
- Continuous delivery improves on continuous integration by helping teams to gain a greater level of certainty that their software will work in production.



Section 4: Deploying applications with CloudFormation

Infrastructure as code

- Infrastructure as code is a method to automate the process of creating, updating, and deleting AWS infrastructure.
- Stand up identical dev/test environments on demand.
- Use the same code to create your production environment that you used to create your other environments.



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

18

As you have seen in this course, nearly everything that you can create in AWS through the AWS Management Console can be created programmatically by using an SDK or the AWS Command Line Interface (AWS CLI). You can use this capability to define your infrastructure as a series of scripts or other code artifacts. This helps you to create, update, and delete your AWS infrastructure automatically and in a repeatable fashion. With this capability, you can create multiple environments—development, testing, staging, and production—from the same code base. This capability is referred to as *infrastructure as code*.

AWS CloudFormation



AWS
CloudFormation

- **Fully managed** service
- Creates, updates, and deletes resources in **stacks**
- **Automates** AWS resource provisioning
- Simplifies the task of **repeatedly** and **predictably** creating groups of related resources that power your applications



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

19

AWS CloudFormation is a fully managed service that provides a common language for you to describe and provision all of the infrastructure resources in your cloud environment. CloudFormation creates, updates, and deletes the resources for your applications in environments called *stacks*.

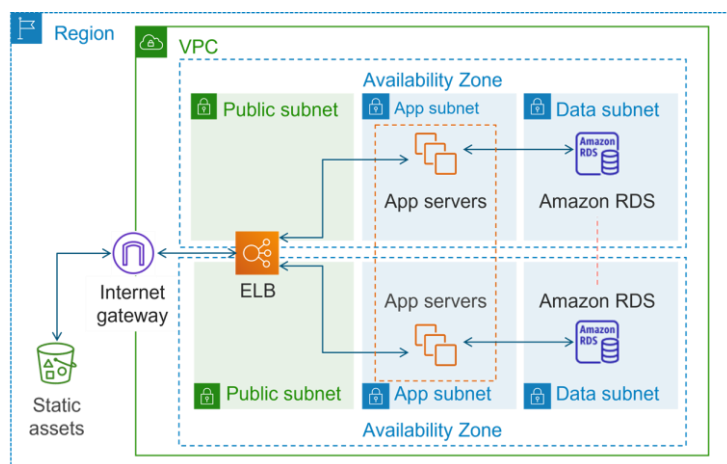
CloudFormation is all about automated resource provisioning—it simplifies the task of repeatedly and predictably creating groups of related resources that power your applications.

CloudFormation supports the infrastructure needs of many types of applications, such as existing enterprise applications, legacy applications, applications built using a variety of AWS resources, and container-based solutions (including those built with AWS Elastic Beanstalk).

For more information about CloudFormation, see the following resources:

- AWS CloudFormation: <https://aws.amazon.com/cloudformation/>.
- AWS CloudFormation FAQs: <https://aws.amazon.com/cloudformation/faqs/>.

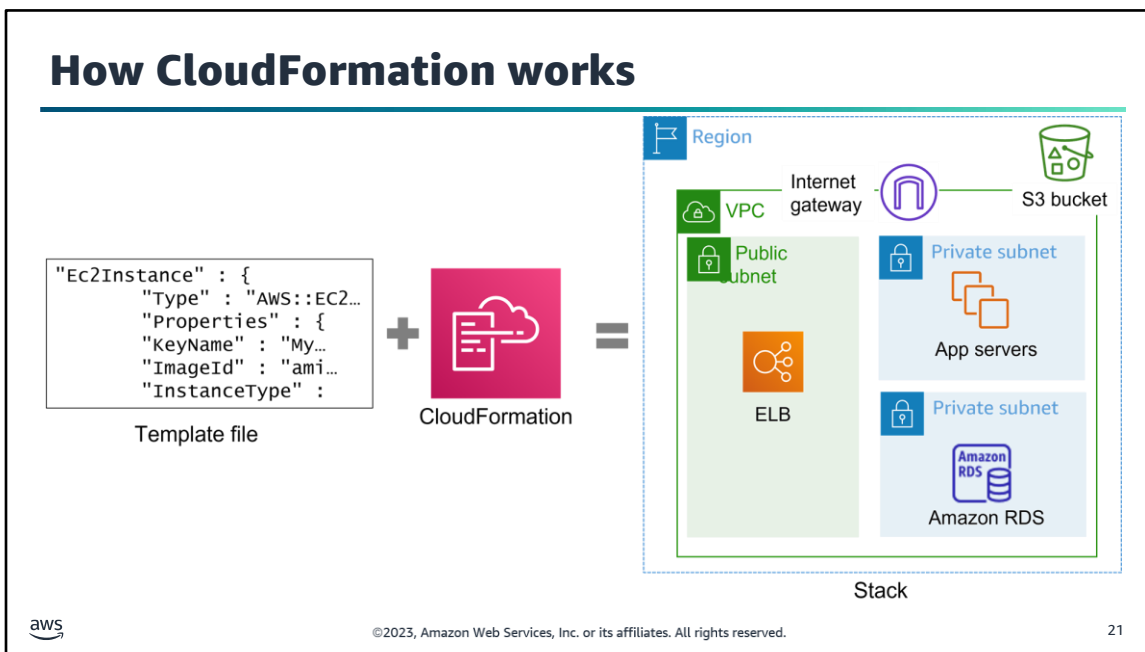
Automated provisioning of AWS resources



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

20

This example shows an application running on EC2 instances within a private (app) subnet, behind a public subnet, in a VPC. The application calls an Amazon RDS database and static assets in the form of an S3 bucket. Imagine the amount of time that it would take to manually deploy and update these services. Automated provisioning with CloudFormation can help streamline this process.

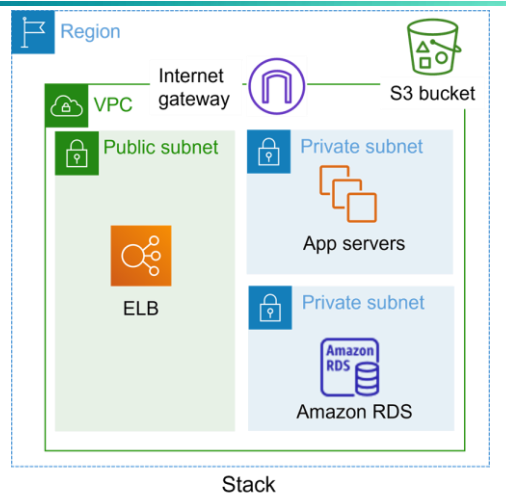


You can automate the provisioning of these AWS resources with CloudFormation. CloudFormation reads *template files*, which provide instructions for what resources need to be provisioned. CloudFormation constructs the resources listed in the template file. The output of this process is your environment, or *stack*. You can create a template that creates a single resource stack or a stack with hundreds of resources.

You can interact with CloudFormation by using the console, AWS CLI, and the AWS SDKs or APIs directly.

CloudFormation stacks

- A CloudFormation *stack* is a unit of deployment.
- Stacks consist of resources that are generated by a template.
- By using templates, you can create and update the collection of resources in stacks.
- The number of stack you deploy may be limited by quotas.
- If you delete a stack, all of the resources in the stack are deleted.



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

22

Stacks are the resources generated by a template. A stack is also a unit of deployment. You can create stacks, update stacks by running a modified template, and delete stacks. When you delete a stack, all of the resources in the stack are deleted by default, though this behavior can be reconfigured and overridden.

When you author templates and create stacks, you might need to consider the quotas (formerly referred to as limits) that AWS has by default. If you know these quotas in advance, you can avoid limitation errors that could require you to redesign your templates or stacks. If you need additional stacks beyond your quota, you can request an increase through the Support Center in the console.

For more information about CloudFormation stacks, see the following resources:

- AWS CloudFormation Quotas:
<https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/cloudformation-limits.html>.
- Working with Stacks:
<https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/stacks.html>.

CloudFormation templates

A CloudFormation template:

- Is a text file
- Is formatted in JSON or YAML
- Is a self-documenting environment
- Designates resources to provision
- Supports the DevOps practice of infrastructure as code

```
"AWSTemplateFormatVersion" : "20...",
"Description" : "My...",
"Resources" : {
  "MyEC2Instance" : {
    "Type" : "AWS::EC2::In...",
    "Properties" : {
      "ImageId" : "ami...",
      "InstanceType" : "t2.mi..."
```

Template file



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

23

A CloudFormation template is a JSON-formatted or YAML-formatted text file that describes your AWS infrastructure. If you provision your environment by using templates, your templates become a form of documentation for your environment.

CloudFormation follows the ECMA-404 JSON standard. The service also supports the YAML Version 1.1 specification with a few exceptions. CloudFormation does not support the following features:

- Binary, omap, pairs, set, and timestamp tags
- Aliases
- Hash merges

CloudFormation templates support the DevOps practice of infrastructure as code because they help you to control your infrastructure through software.

CloudFormation template structure

```
{  
  "AWSTemplateFormatVersion" : "version date",  
  "Description" : "JSON string",  
  "Metadata" : {template metadata},  
  "Parameters" : {set of parameters},  
  "Mappings" : {set of mappings},  
  "Conditions" : {set of conditions},  
  "Transform" : {set of transforms},  
  "Resources" : {set of resources},  
  "Outputs" : {set of outputs}  
}
```

2010-09-09
is the latest
version

Required



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

24

This slide contains an example of a CloudFormation template structure and its sections. Templates include the following major sections:

- *Format version*: The CloudFormation template version that the template conforms to, which identifies the capabilities of the template. The latest template format version is 2010-09-09 and is currently the only valid value.
- *Description*: A text string that describes the template.
- *Metadata*: Objects that provide additional information about the template.
- *Parameters*: Values to pass to your template at runtime when you create or update a stack.
- *Mappings*: A mapping of keys and associated values, which you can use to specify conditional parameter values, similar to a lookup table.
- *Conditions*: Conditions control whether certain resources are created or whether certain resource properties are assigned a value during stack creation or update. For example, you could conditionally create a resource that depends on whether the stack is for a production or test environment.
- *Transform*: For serverless (that is, Lambda based) applications, this specifies the version of the AWS SAM to use. When you specify a transform, you can use AWS SAM syntax to declare resources in your template. The model defines the syntax that you can use and how it is processed. You will learn about AWS SAM in another section of this module.
- *Resources*: Specifies the stack resources and their properties, such as an EC2 instance or S3 bucket. **This is the only required section.**
- *Outputs*: Describes the values that are returned whenever you view your stack's properties.

You don't have to list your resources in the template in the exact order of creation. You can use the *DependsOn* attribute to specify the order in which CloudFormation will create the resources. In this way, you can build a sequence of events, such as creating a database server before a web server.

For more information, see the following resources:

- Template Anatomy: <https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/template-anatomy.html>.
- Resource Attribute Reference: <https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/aws-product-attribute-reference.html>.
- Format Version: <https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/format-version-structure.html>.

CloudFormation template example: Description

```
{  
  "AWSTemplateFormatVersion": "2010-09-09",  
  
  "Description": "AWS CloudFormation Sample Template ElasticBeanstalk_Simple:  
  Configure and launch an Elastic Beanstalk application that connects to an  
  Amazon RDS database instance. Monitoring is set up on the database."  
  
  "Parameters": {  
    "DBUser": {  
      "NoEcho": "true",  
      "Type": "String",  
      "Description": "Test database admin account name"  
    },  
  },  
}
```



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

25

The description section is a text string that describes the template.

CloudFormation template example: Parameters

```
{
  "AWSTemplateFormatVersion": "2010-09-09",

  "Description": "AWS CloudFormation Sample Template Config: This template
demonstrates the use of AWS Config resources. **WARNING** You will be billed
for the AWS resources used if you create a stack from this template.",

  "Parameters": {
    "Ec2VolumeAutoEnableIO": {
      "Type": "String",
      "Default": "false",
      "AllowedValues": ["false", "true"]
    }
  },
```



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

26

Parameters are values to pass to your template at runtime when you create or update a stack.

CloudFormation template example: Resources

```
"AWSTemplateFormatVersion": "2010-09-09",  
"Resources": {  
  "Ec2Volume": {  
    "Type": "AWS::EC2::Volume",  
    "Properties": {  
      "AutoEnableIO": {"Ref": "Ec2VolumeAutoEnableIO"},  
      "Size": "5",  
      "AvailabilityZone": {"Fn::Select": [0, {"Fn::GetAZs": ""}]},  
      "Tags": [{  
        "key": {"Ref": "Ec2VolumeTagKey"},  
        "value": "Ec2VolumeTagValue"  
      }]  
    }  
  }  
}
```



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

27

The resources section is the only required section of the CloudFormation template. This section specifies the stack resources and their properties, such as an EC2 instance or S3 bucket. This example describes an EC2 volume resource.

Section 4 key takeaways



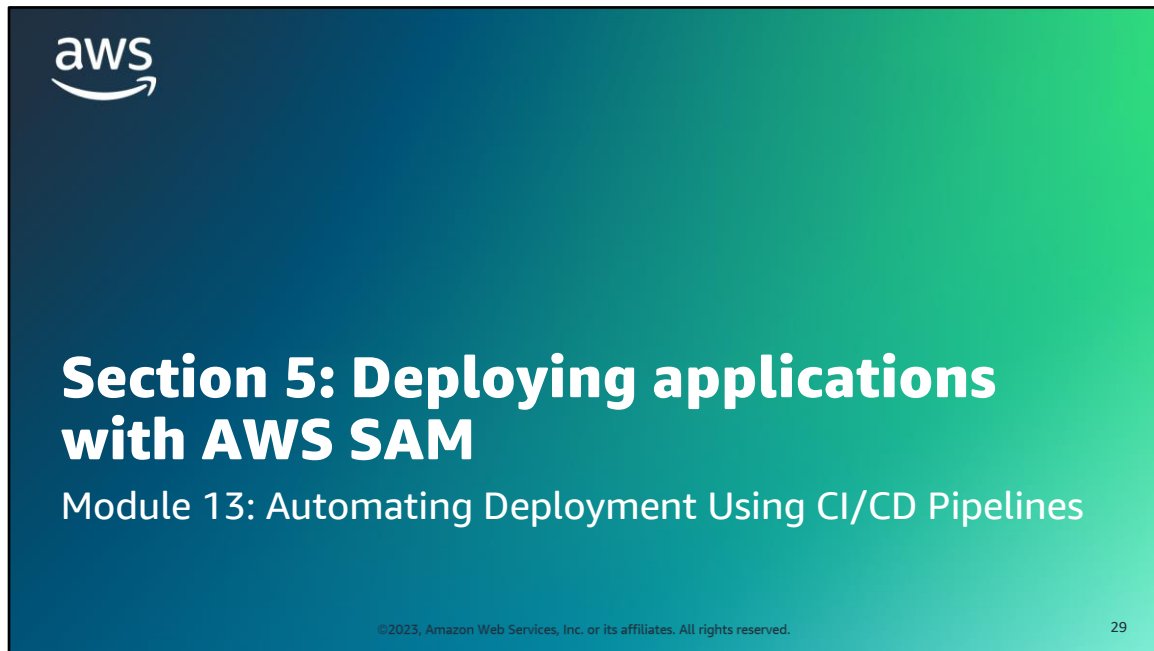
- CloudFormation is a **fully managed** service that automates AWS resource provisioning.
- CloudFormation uses JSON-formatted or YAML-formatted **template files** to provide resource provisioning instructions.
- A **stack** is a unit of deployment that you can **create** and **update** by using templates. When a stack is **deleted**, resources in the stack are also **deleted**.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

28

The following are the key takeaways from this section of the module:

- CloudFormation is a fully managed service that automates AWS resource provisioning.
- CloudFormation uses JSON-formatted or YAML-formatted template files to provide resource provisioning instructions.
- A stack is a unit of deployment that you can create and update by using templates. When a stack is deleted, resources in the stack are also deleted.



Section 5: Deploying applications with AWS SAM

AWS Serverless Application Model (AWS SAM)

- Open-source framework for building serverless applications
- Use to build templates that define your serverless applications
- Deploy your template with CloudFormation
- Integrates with CI/CD tools
- Provides an invocation environment locally



AWS SAM



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

30

With serverless applications, you must provide everything that is needed to deploy a function—the code, any necessary dependencies, and the blueprint to set up the infrastructure. However, you can't create the environment that you specified in your blueprint locally. You don't connect to a specific server from which you debug code. You need the ability to deploy your stack to each AWS account. This is where an application framework such as AWS SAM is useful.

AWS SAM is an open-source framework for building serverless applications. AWS SAM is an extension of CloudFormation, so you get the reliable deployment capabilities of CloudFormation.

You can use AWS SAM to build templates that define your serverless applications. You then deploy your AWS SAM template with CloudFormation.

AWS SAM has two components:

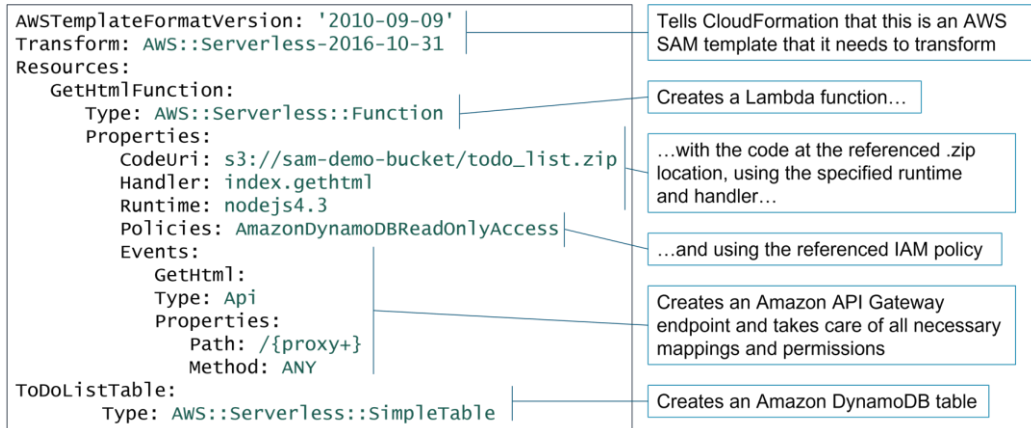
- *Template specification:* This specification defines your serverless application. It provides a shorthand syntax to describe the functions, APIs, permissions, configurations, and events that make up a serverless application. Use an AWS SAM template file to operate on a single, deployable, versioned entity that is your serverless application.
- *CLI:* With this tool, you can build serverless applications that are defined by AWS SAM templates. The CLI provides commands to verify that AWS SAM template files are written according to the specification. Commands are also available to invoke Lambda functions locally, step-through debug Lambda functions, package and deploy serverless applications to the AWS Cloud, and so on.

AWS SAM has deep integration with CI/CD tools. You can use AWS SAM with CodeBuild, CodeDeploy, and CodePipeline to build a deployment pipeline for your serverless applications. AWS SAM provides an invocation environment similar to Lambda, which is helpful for step-through and debugging of code. This can help you to catch issues upfront rather than later in the process.

For more information, see the following resources:

- AWS Serverless Application Model: <https://aws.amazon.com/serverless/sam/>.
- What is the AWS Serverless Application Model (AWS SAM)?:
<https://docs.aws.amazon.com/serverless-application-model/latest/developerguide/what-is-sam.html>.
- The AWS SAM specification, the code that translates AWS SAM templates into CloudFormation stacks, general information about the model, and examples of common applications:
<https://github.com/aws-labs/serverless-application-model>.

AWS SAM template



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

31

The AWS SAM template follows the CloudFormation template format, with the following differences:

- The transform declaration `Transform: AWS::Serverless-2016-10-31` is required, because it identifies the template as an AWS SAM template.
- The `globals` section is unique to AWS SAM and is used to define properties that are common to all of your serverless functions and APIs.
- The `resources` section contains resources for both CloudFormation and AWS SAM.
- The `parameters` section is used to declare objects and present additional prompts to the user.

The slide displays an example of an AWS SAM template with the following information:

- The transform declaration of AWS CloudFormation specifies that this is an AWS SAM template.
- In the `resources` section, `AWS::Serverless::Function` tells CloudFormation to create a Lambda function.
- The `properties` section references the .zip file, handler, and runtime to be used.
- The `policies` section provides the IAM policy to be referenced for the function.
- The template also includes commands to create an Amazon API Gateway endpoint. The AWS SAM transform will perform the mappings and permissions.
- The last part of the template creates an Amazon DynamoDB table.

For more information about AWS SAM template anatomy, see

<https://docs.aws.amazon.com/serverless-application-model/latest/developerguide/sam-specification-template-anatomy.html>.

Section 5 key takeaways



- AWS SAM provides the capability to deploy your stack to **each AWS account**.
- As an **extension** of CloudFormation, AWS SAM provides the reliable deployment capabilities of CloudFormation.
- AWS SAM has deep **integration** with CI/CD tools such as:
 - AWS CodeBuild
 - AWS CodeDeploy
 - AWS CodePipeline

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

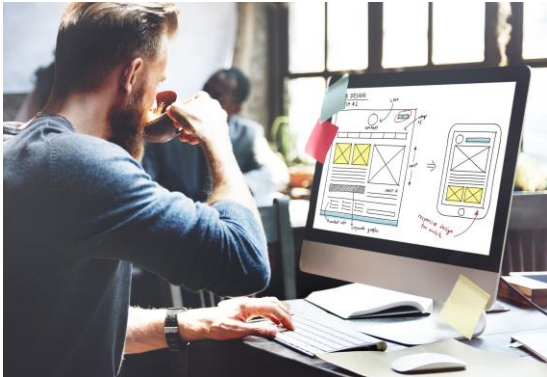
32

The following are the key takeaways from this section of the module:

- AWS SAM provides the capability to deploy your stack to each AWS account.
- As an extension of CloudFormation, AWS SAM provides the reliable deployment capabilities of CloudFormation.
- AWS SAM has deep integration with CI/CD tools such as the following:
 - AWS CodeBuild
 - AWS CodeDeploy
 - AWS CodePipeline

Lab 13.1: Automating Application Deployment Using a CI/CD Pipeline





©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

33

You will now complete Lab 13.1: Automating Application Deployment Using a CI/CD Pipeline.

Lab: Scenario

Now that the café website is in production, Sofía needs to centralize the website code and add version control. She also wants to automate the process of updating the website.

Sophia plans to use a code repository to store the website code. Mateo, the AWS consultant, suggested using CodePipeline to support automating updates to the website.



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

34

In this lab, you will:

Lab: Tasks

1. Preparing the development environment
2. Creating a code repository
3. Creating a pipeline to automate website updates
4. Cloning a repository
5. Exploring the Git integration with the IDE
6. Pushing the café website code to the code repository



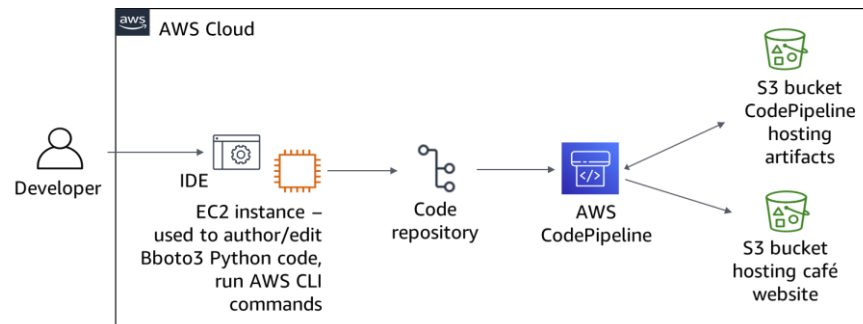
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

35

In this lab, you will complete the following tasks:

1. Preparing the development environment
2. Creating a code repository
3. Creating a pipeline to automate website updates
4. Cloning a repository
5. Exploring the Git integration with the IDE
6. Pushing the café website code to the code repository

Lab: Final product



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

36

The diagram summarizes what you will have built after you complete the lab.



The image is a promotional banner for an AWS lab. It features a dark blue header with the AWS logo and a stopwatch icon indicating a duration of approximately 60 minutes. Below the header is a photograph of a wooden coffee scoop filled with coffee beans, with a few beans scattered on a dark surface next to a teal ceramic coffee cup. To the right of the photograph, the text 'Begin Lab 13.1: Automating Application Deployment Using a CI/CD Pipeline' is displayed in a large, bold, white font. At the bottom of the banner, there is a teal bar containing the copyright notice '©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.' on the left and the page number '37' on the right.

aws ~ 60 minutes

**Begin Lab 13.1:
Automating
Application
Deployment Using
a CI/CD Pipeline**

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved. 37

It is now time to start the lab.

Lab debrief: Key takeaways

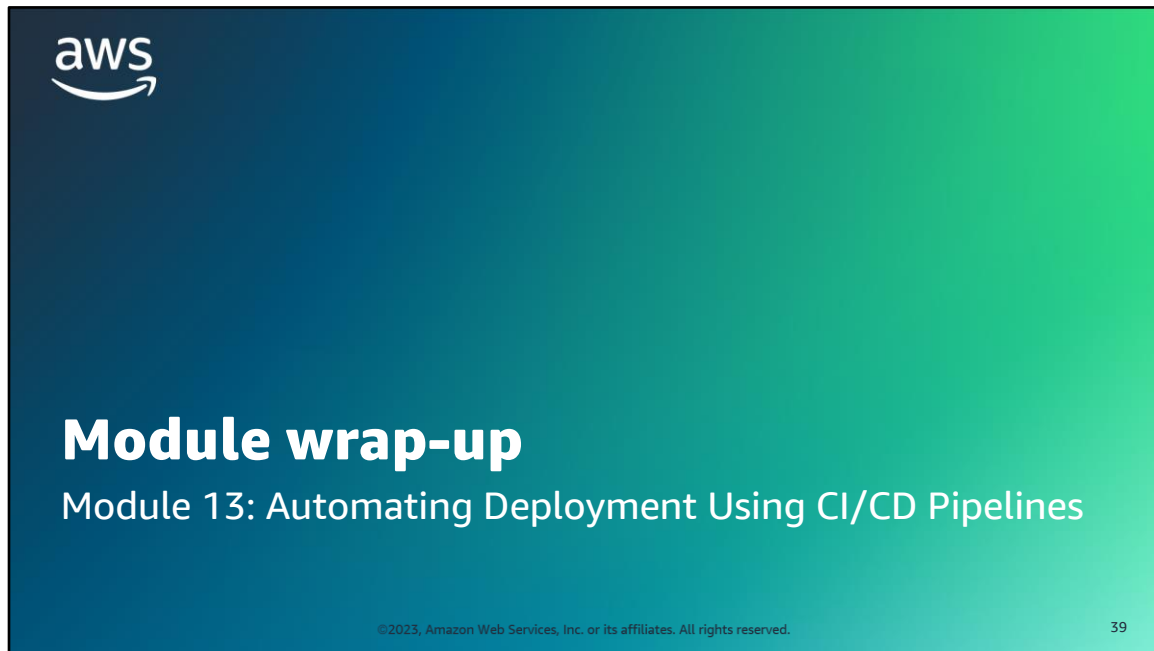


aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

38

After you complete the lab, your educator might choose to lead a conversation about the key takeaways from the lab.



It's now time to review the module and wrap up with a knowledge check and discussion of a practice certification exam question.

Module summary

In summary, in this module, you learned how to do the following:

- Describe DevOps
- Recognize AWS code services for CI/CD
- Describe how CloudFormation is used to deploy applications
- Describe how AWS SAM is used to deploy serverless applications



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

40

In summary, in this module, you learned how to:

- Describe DevOps
- Recognize AWS code services for CI/CD
- Describe how CloudFormation is used to deploy applications
- Describe how AWS SAM is used to deploy serverless applications

Complete the knowledge check



aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

41

It is now time to complete the knowledge check for this module.

Sample exam question

A developer has reached the account quota for the number of CloudFormation stacks in a Region.

How could they increase the quota?

Identify the key words and phrases before continuing.

The following are the key words and phrases:

- Account quota
- CloudFormation stacks



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

42

It is important to fully understand the scenario and question being asked before even reading the answer choices. Find the keywords in this scenario and question that will help you find the correct answer.

Sample exam question: Responses

A developer has reached the **account quota** for the number of **CloudFormation stacks** in a Region.

How could they increase the quota?

Choice	Response
A	Use the AWS Command Line Interface (AWS CLI).
B	Send an email to quotas@amazon.com with the subject "CloudFormation."
C	Use the Support Center in the AWS Management Console.
D	All service quotas are fixed and cannot be increased.

 ©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved. 43

Now that we have bolded the keywords in this scenario, let us look at the answers.

Sample exam question: Answer

The correct answer is C.

Choice	Response
A	Use the AWS Command Line Interface (AWS CLI).
B	Send an email to quotas@amazon.com with the subject "CloudFormation."
C	Use the Support Center in the AWS Management Console.
D	All service quotas are fixed and cannot be increased.

 ©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved. 44

Look at the answer choices, and rule them out based on the keywords that were previously highlighted.

The correct answer is C. Use the Support Center in the AWS Management Console.

Using the Support Center in the AWS Management Console, users can request a quota increase for service quotas that do not have a fixed maximum capacity.

Additional resources

- Whitepapers:
 - [Introduction to DevOps on AWS](#)
 - [Infrastructure as Code](#)
 - [Overview of Deployment Options on AWS](#)
 - [Practicing Continuous Integration and Continuous Delivery on AWS](#)
- Tutorial:
 - [Build a Serverless Web Application](#)



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

45

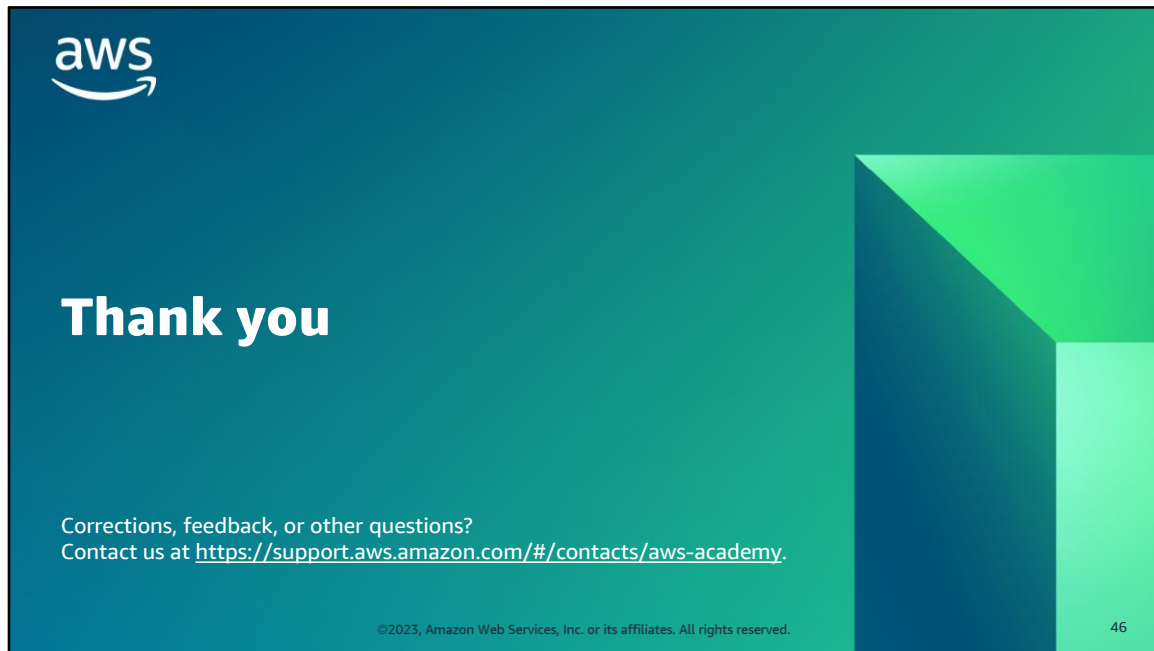
To learn more about the topics covered in this module, you might find the following additional resources helpful:

Whitepapers:

- Introduction to DevOps on AWS:
https://d1.awsstatic.com/whitepapers/AWS_DevOps.pdf.
- Infrastructure as Code:
<https://d1.awsstatic.com/whitepapers/DevOps/infrastructure-as-code.pdf>.
- Overview of Deployment Options on AWS:
<https://d1.awsstatic.com/whitepapers/overview-of-deployment-options-on-aws.pdf>.
- Practicing Continuous Integration and Continuous Delivery on AWS:
<https://d1.awsstatic.com/whitepapers/DevOps/practicing-continuous-integration-continuous-delivery-on-AWS.pdf>.

Tutorial:

- Build a Serverless Web Application: <https://aws.amazon.com/getting-started/projects/build-serverless-web-app-lambda-apigateway-s3-dynamodb-cognito/>.



Thank you for completing this module.